Design
Tips
Changes
Look and Feel
Windows
Components
Content
Accessibility
Layout
Menus and actions
Navigation
Presentation
Selection and input
Status
Text entry
System experiences
App icons
Complications
Controls
Live Activities
Notifications
Stickers
Top Shelf
Widgets
Inclusive design
Patterns
Foundations

Widgets

A unique glance and display a small amount of timely, relevant information from your app or game so people can see it at a glance in additional contexts.

Widgets display content and offer specific functionality without requiring people to open your app or game. They can also appear in new and prominent locations on their devices, quickly enhancing the time people find value from their apps.

The system can display widgets in different places depending on a person's device. In iOS and iPadOS, widgets appear on the Home Screen, in Today View, and in the App Library. In watchOS, people can find widgets on the Dock and in Smart Stack. In macOS, widgets appear in Notification Center and on the Desktop.

People can access widget gallery from the same place where widgets can appear by going into edit mode — for example, by tapping-hold on the Home Screen. The widget gallery contains all the widgets that people can add to their Home Screen, Today View, and Notification Center, including widgets that people add to the Dock and in Smart Stack. In watchOS, widgets appear in a list in a focus. In macOS, the gallery also includes iPhone widgets from devices that people use with iPhone Mirroring. In iOS and iPadOS, people can also add widgets to the Lock Screen on iPhone and iPad. A suggested widget doesn't stay in the Smart Stack unless people choose to keep it. For developer guidance, see Increasing the visibility of widgets in Smart Stacks.

Widgets come in different sizes, ranging from small accessory widgets on the Lock Screen in iOS and iPadOS and on watchOS to extra-large in iPadOS and macOS.

The following table shows the available widget sizes for each platform:

Widget size	iPhone	iPad	Apple Watch	Mac
System small	Home Screen, Today View, Lock Screen, Smart Stack	Home Screen, Today View, Lock Screen, Smart Stack	Dock, Smart Stack	Desktop and Notification Center
System medium	Home Screen and Today View	Home Screen and Today View	Smart Stack	Desktop and Notification Center
System large	Home Screen and Today View	Home Screen and Today View	Smart Stack	Desktop and Notification Center
System extra-large	No	Home Screen and Today View	No	Desktop and Notification Center
Accessory circular	Lock Screen	Lock Screen	No	No
Accessory rectangular	Lock Screen	Lock Screen	Smart Stack	No
Accessory inline	Lock Screen	Lock Screen	Smart Stack, Below time on watch face	No

In each size, display only the information that's directly related to the widget's core purpose. A large widget, you can display more data — or more detailed visualizations of the data — than a small widget. For example, a small widget that shows the current temperature in Celsius might display a person's upcoming events. In each size, the widget remains centered on events while expanding the range of information as the size increases.

Use a consistent layout and visual design across all widget sizes. Avoid using completely different layouts or color schemes for each widget size. It's more important to make each widget in the set that works best for the content you want to display than it is to provide the widget in all sizes.

Aim to create a widget that gives people quick access to the content they want. Provide a tap-to-action experience that's fast and lightweight, rather than a complex, multi-step flow. Any time it can help, offer a single-step tap-to-action experience. For example, if people use your app to keep a shopping list, a widget that helps people add items to the list is more valuable than a widget that only shows the list.

Use dynamic information that changes throughout the day. If a widget's content never appears to change, people may not keep it in a prominent position. Although widgets don't update from minute to minute, it's important to find ways to keep that content fresh to invite people to interact.

Look for opportunities to surprise and delight. For example, you can offer a special visual treatment for your calendar widget to display on meaningful occasions, like birthdays or holidays.

Let people know when authentication adds value. If your widget provides additional functionality when authenticated is logged in to your app, make sure people know that. For example, an app that shows upcoming reservations might include a message like "Sign in to view reservations" when people are signed out.

Updating widget content

To remain relevant and useful, widgets periodically refresh their information. Widgets don't refresh their content all at the same time, and the timing depends on factors such as how frequently people use the widget.

Keep your widget up to date. Finding the appropriate update frequency for your widget requires balancing several factors. Consider how quickly the information in your widget changes, how the user uses the widget, how critical a widget that helps people make decisions that depend on timely information is an hourly basis, even though data collection changes constantly. If people are using your app or game, you can also use background tasks to update your app's data, which can describe when the data was last updated. For developer guidance, see Keeping a widget up to date.

Use system functionality to refresh dates and times in your widget. Widget updates frequently in iOS, so you can preserve some of your update opportunities by letting system code update the date and time information.

Encourage the system to display or elevate the position of your watchOS widget in the Smart Stack. The system can display a widget in the Smart Stack when it's relevant to the person's day. Relevance can be boosted based on specific ongoing system actions, like a workout. For additional guidance, see Keeping Smart Stacks fresh.

Show current updates. When you determine the update frequency that fits with the data you're displaying, make sure people see the most current information.

Use animated transitions to bring attention to key updates. By default, iOS and iPadOS apply animated content updates. Use standard and custom animations with a duration of up to two seconds to draw people's attention to important information and to enhance the overall experience.

Offer Live Activities to show real-time updates. Widgets don't show real-time information. If your app offers people a track they progress on a timer, it can be useful for a limited amount of time with frequent updates, consider offering Live Activities in your app. Widgets and Live Activities can share a common design language and code base, so people can easily find the right tool for the job. Widgets can't update and Live Activities in tandem and issue code and design changes to both Widgets and Live Activities. For design guidance on Live Activities, see Live Activities. For developer guidance, see ActivityKit.

Configuring widgets

In some cases, people need a widget to ensure it displays the information that's most useful to them. For example, people choose a stock symbol for a Stocks widget. In contrast, a Weather widget can show the current weather in people's current location, so people don't need to customize them.

Make editable widgets easy for people to customize. If your widget is editable, avoid requiring people to open your app or game to edit the widget. Instead, let people edit the widget in place. To design a widget editing user interface for your widget because the system automatically generates it for you, for developer guidance, see Making a configurable widget.

Adding interactivity to widgets

People can tap a widget to launch its corresponding app. In iOS, iPadOS, macOS, and watchOS, widgets can offer multiple deep links that open the app and can include parameters that specify the information the app displays. For example, a Dining widget might offer a separate deep link for each nearby restaurant. Note that while a accessory widget can launch the app, it can't display the information in the app. Furthermore, the system displays the widget's name in the app's Settings, so people can easily find the app's settings. For developer guidance, see Linking to specific app views from a widget, and Configuring a widget's settings. Note that iOS and iPadOS also offer widgets that can't open the app at all, for example, a Calculator widget.

Offer simple, relevant functionality in a widget, reserving complex functionality for your app. Widgets can offer some, but not all, functionality a person can find in their app. For example, a Calendar widget can let people add events, but not edit them.

Ensure that a widget interaction opens your app in the right location. When people interact with your widget, it's important to open your app to the most relevant location. For example, if a person taking people escape to the relevant area in the app, and instead takes them to a place where you offer details and actions that directly relate to the widget's content. For example, when people tap a Dining widget, the app should open to a page that displays information about the current restaurant.

Provide options for interactivity that complement, not duplicate, in iOS, iPadOS, macOS, and watchOS, widgets can offer multiple deep links that open the app and can include parameters that specify the information the app displays. For example, a Dining widget might offer SwiftUI, Safari, and Maps — might make sense for your content, but can't duplicate app the layouts in your widgets. Pay attention to the size of targets and make sure people can tap or click the right area of a widget to open the app. In macOS, people can also interact with a widget's UI, use a confirmation popup to avoid unintended interactions. Note that iOS accessory widgets offer only one tap target.

Interface design

Widgets use vibrant colors, rich images, and clear, crisp text that's easy to read at a glance. A unique, beautiful widget not only provides useful information, it can encourage people to feature it on their Home Screen.

Help people recognize your widget by including design elements found in your app's UI. Design elements like brand colors, typeface, and stylized glyphs can make a widget more recognizable. For example, a Dining widget might use a distinctive font when crediting its useful information or making your widget look out of place in its content.

Notes

Widget offers access to Notes app's content in iMessage

Widget uses Notes app's typeface, typeface, and stylized glyphs can make a widget more recognizable. For example, a Dining widget might use a distinctive font when crediting its useful information or making your widget look out of place in its content.

Consider carefully before displaying a logo, wordmark, or app name in your widget. When you think it would clearly help people recognize the content, people and people, people and people, people and people icon to help them recognize your widget. Note that iOS and iPadOS automatically place a small version of your app's icon in the upper-left corner of the Home Screen, so it's always clear that people created from multiple sources — it may make sense to include a small logo in the top-right corner to subtly identify the app that provides the widget.

Aim for a consistent density of information. When content appears scattered, the widget can seem unnecessary when contrast is low, details are difficult to perceive. If you have lots of content, it's better to pick the most important information to display than to try to fit it all. Leave some space to ensure that content is not jarring and that people can easily read the content. Avoid using a dense layout that's difficult to read, and don't let the widget's content look like it's hanging out in the middle of a large, empty area. Consider using a dense layout that's difficult to read, and don't let the widget's content look like it's hanging out in the middle of a large, empty area.

Use color judiciously. Beautiful colors draw the eye, but they're least effective when they prevent people from attending a widget's information at a glance. Use color to enhance a widget's appearance and to emphasize key data, but don't use it to the point that it's difficult to read. In iOS, iPadOS, macOS, and watchOS, widgets can use a vibrant color to add a pop of color to the widget's content. Avoid retrieving your widget's content from a remote server.

Avoid retrieving your widget's appearance within your app. If your app displays an element that's unique to your app's appearance, it's better to draw it in your app than to try to retrieve it. Widgets request efficiently when they consult with it, they people may be less likely to try other ways to interact with such an element in your app because they expect it to behave like a widget.

Scaling content and using margins and padding

Widgets are designed to fit the screen areas of different devices and accessories. Ensure that your widget's content looks great in all sizes by using the following guidelines.

Use dynamic layout to scale your content and ensure it looks right on a variety of sizes. Avoid hard-coded values, instead, use dynamic layout to ensure that your widget's content fits the content you design for large devices. In iPadOS, the system renders your widget at a large size, for example, and then scales it down to fit the Home Screen. Dynamic layout makes it unnecessary for various devices and sizes, for developer guidance, see Backing the widget's appearance with a timeline.

Use the system-provided margins and padding to ensure the content of the widget. To ensure that your content looks right and fits in a widget's available screen, use a SwiftUI container to apply the correct corner radius. For developer guidance, see ContainerRelativeShape.

Note

In iOS, widgets support Dynamic Type sizes from Large to AX5 when you use Text to choose a system font or custom. Use the preferred font modifier. For more information about Dynamic Type sizes, see Typography.

Displaying text in widgets

Consider using the system font, text styles, and SF Symbols. Using the system font helps your widget's text look crisp and easy to read at a glance, while making it easy for you to display content in a variety of weights, styles, and sizes. Use the SF Symbols in style and weight that matches the text you pair it with. For example, if you use the system font's bold weight, you can use the bold version of a symbol that's well suited to use a custom font for the larger text is a unique set of Pro for the text weight. For guidance, see Typography and SF Symbols.

Use a font size that's large enough to be easily read. Text that's too small is difficult to read. Text at a minimum of 11 points is large enough for most people to read comfortably.

Always use text dimensions in a widget's layout that don't scale with it. In particular, don't underlay text — doing so prevents dynamic text from scaling your content.

Supporting different appearances and modes

For every appearance, a unique, beautiful widget not only provides useful information, it can encourage people to feature it on their Home Screen. Widgets on the contact in iOS Home Screen widgets can look different. For example:

- Color scheme from color scheme to tinted, monochrome colors.
- Images can from full color images, to monochrome images, to symbols and glyphs only.

For example, a small system widget appears as follows:

- On the Home Screen, iPad, iPhone, the widget takes on a rich, full color appearance that supports light and dark appearances.

- On the Lock Screen iPad, the widget takes on a vibrant appearance.

- On the Lock Screen of iPhone in Standby, the widget appears scaled up in size, and uses the system-provided vibrant appearance. Because the widget's content is a timeline, Standby's high-mode weeks widget contains a monochromatic red hue.

Standby

Night mode

- In Notification Center in macOS, the widget uses rich, full colors and supports both light and dark appearances.

- On the Desktop on Mac, the widget uses rich, full colors when people interact with it. When people interact with widgets, the widget uses vibrancy and a blurred background to remain.

Full color appearance Tinted appearance

Similarly, a rectangular accessory widget appears as follows:

- On the Lock Screen of iPhone and iPad, it takes on a vibrant appearance.

- On Apple Watch, the widget can appear as a custom complication in both full color and tinted appearances, and it can also appear in the Smart Stack.

Vibrant Lock Screen Tinted Watch face, Smart Stack

The following table lists the available rendering modes for various types and sizes of widgets. For more information, see Preparing widgets for additional platforms, watchos, and accessory.

Widget size	Full color	Accented	Vibrant (included in vibrant)
System small	Yes	Yes	Yes
System medium	Yes	Yes	Yes
System large	Yes	Yes	Yes
System extra-large	Yes	Yes	Yes
Accessory circular	No	Yes	Yes
Accessory rectangular	No	Yes	Yes
Accessory inline	No	Yes	Yes

Support Dark Mode. Ideally, a widget looks great in both the light and dark appearances. In general, avoid displaying dark text on a dark background in the dark appearance, or light text on a light background in the light appearance. When you use the semantic system colors for text and vibrancy, the system ensures that your content looks great in both modes. Widgets can offer different appearance by putting color variants in your asset catalog. For guidance, see Asset Mode, for developer guidance, see Asset management and Supporting Dark Mode in your interface.

Support Standby and Night mode. In Standby, the system displays key small system family widgets side by side, scaled up so they fill the Lock Screen. Widgets that appear in Standby typically don't use full images or color to convey meaning but instead make use of the additional space to show more information, or information that people can see from far enough away to read glance. Alternatively, for people who sleep with the Apple Watch, the system may insert widgets into a special Night mode. For guidance, see Preparing widgets for additional platforms.

Current usage Accented usage

In Night mode, the system applies a red hot to widgets.

Night mode

Adjust colors and images for the vibrant rendering mode. The system renders widgets on the Lock Screen and on the Desktop in a vibrant style. In this mode, the system applies a tint color to widgets using an image determine the strength of the tint. Vibrant rendering affects the appearance of widgets, for example, the system applies a tint color to the widget's content. For guidance, see Preparing widgets for additional platforms. For developer guidance, see Applying a tint to a widget.

Use full color to highlight images. In iOS and iPadOS, you can specify images in a widget to remain full color. For example, a photo widget might let people see their photos in full color. Note that full color images need to have visible dimensions that fills the shape of the widget.

Convey meaning without relying on specific colors to represent information. Sometimes may choose a color that changes the picture of the information. For example, in watchOS, the system may insert widgets depending on the current time of day.

Previews and placeholder content

Provide a preview that shows the widget's value, highlighting your widget's capabilities. Let people understanding the experiences which widget brings to your app provide — helps people make an informed decision. You can also make it easier for people to find, and the data they're going to be in a widget, display relevant data instead.

Design placeholder content that helps people recognize your widget. An instead widget appears in a placeholder state while it's loading. Use a simple, generic representation of your widget's appearance to quickly convey functionality and layout. Avoid using a blank or empty state. Instead, let people know what to expect. For example, you can leverage of different widths to suggest lines of text, and of the square or shape of glyphs and images.

Write a succinct description of your widget. The widget gallery displays descriptions that help people understand the purpose of each widget. Keep your widget's description succinct and to the point, which aids in its clarity. Take into account whether a phrase and forward to a noun or "Keep track of your upcoming events and meetings." Avoid including unnecessary phrases that don't add value, such as "Widget that shows" or "Displays." Avoid ending with "Widget." For advanced tips, see Adding a widget to a gallery.

Group your widget's core together, and provide a single description. If your widget is available in multiple sizes, group the core together so people don't think each one is a different widget. Provide a single description of your widget — regardless of how many sizes you offer — that describes the core functionality. For example, a Calendar widget can provide a single description of its core content and functionality.

Consider omitting the text from widgets. After people choose your app in the widget gallery, or Add Widget, the system displays your widgets you offer. You can opt to create a list for people to help narrow down your found.

Platform considerations

No additional considerations for macOS that supported in tvOS or visionOS.

iOS, iPadOS

Widgets on the Lock Screen are functionally similar to watch complications and follow design principles. Widgets on the Lock Screen provide people with quick, glanceable access to information in your app. Lock Screen widgets and don't treat it only as an additional way for people to launch into your app. Additionally, the system rendering mode that widgets on the Lock Screen use is vibrant, which means that the system applies a tint color to the widget's content. Design to communicate information without relying on color only. In many cases, a Widget's complications also works well for widgets on the Lock Screen and vice versa, so consider supporting both.

Text styles on the widgets on the Lock Screen in three different shapes, so they look for appears above the clock, and an circular and rectangular shapes that appear below the clock.

Support Always On display on iPhone. Devices with Always On display render widgets on the Lock Screen with reduced screen brightness. Use design to ensure that your widget's content is easy to read in this mode. For additional guidance, see Preparing widgets for additional platforms.

Support Always On display on iPhone. Devices with Always On display render widgets on the Lock Screen with reduced screen brightness. Use design to ensure that your widget's content is easy to read in this mode. For additional guidance, see Preparing widgets for additional platforms.

On display, and make sure your content is legible.

For developer guidance, see Creating accessory widgets and watch complications, WidgetRenderingMode, and vibrant.

watchOS

Provide a colorful background that conveys meaning. By default, widgets in the Smart Stack use a black background. Consider using a custom color that provides additional meaning. For example, the Stocks app uses a red background for falling stock values and a green background if a stock's value rises.

Specifications

As you design your widgets, use the following values for guidance.

iOS widget dimensions

Screen size (portrait, pt)	Small (pt)	Medium (pt)	Large (pt)	Circular (pt)	Rectangular (pt)	In
430x932	170x170	364x170	364x382	76x76	172x76	2
428x926	170x170	364x170	364x382	76x76	172x76	2
414x896	169x169	360x169	360x379	76x76	160x72	2
414x736	159x159	348x157	348x357	76x76	170x76	2
393x852	158x158	338x158	338x354	72x72	160x72	2
390x844	158x158	338x158	338x354	72x72	160x72	2
375x812	155x155	329x155	329x345	72x72	157x72	2
375x667	148x148	321x148	321x324	68x68	153x68	2
360x780	155x155	329x155	329x345	72x72	157x72	2
320x568	141x141	292x141	292x311	N/A	N/A	N

iPadOS widget dimensions

Screen size (portrait, pt)	Target	Small (pt)	Medium (pt)	Large (pt)	Extra large (pt)
768x1024	Canvas	141x141	305.5x141	305.5x305.5	634.5x305.5
	Device	120x120	260x120	260x260	540x260
744x1133	Canvas	141x141	305.5x141	305.5x305.5	634.5x305.5
	Device	120x120	260x120	260x260	540x260
810x1080	Canvas	146x146	320.5x146	320.5x320.5	669x320.5
	Device	124x124	272x124	272x272	568x272
820x1180	Canvas	155x155	342x155	342x342	715.5x342
	Device	136x136	300x136	300x300	628x300
834x1112	Canvas	150x150	327.5x150	327.5x327.5	682x327.5
	Device	132x132	288x132	288x288	600x288
834x1194	Canvas	155x155	342x155	342x342	715.5x342
	Device	136x136	300x136	300x300	628x300
954x1373 *	Canvas	162x162	350x162	350x350	726x350
	Device	162x162	350x162	350x350	726x350
970x1389 *	Canvas	162x162	350x162	350x350	726x350
	Device	162x162	350x162	350x350	726x350
1024x1366	Canvas	170x170	378.5x170	378.5x378.5	795x378.5
	Device	160x160	356x160	356x356	748x356
1192x1590 *	Canvas	188x188	412x188	412x412	860x412
	Device	188x188	412x188	412x412	860x412

* When Display Zoom is set to More Space.

watchOS widget dimensions

Apple Watch size	Size of a widget in the Smart Stack (pt)
40mm	152x69.5
41mm	165x72.5
44mm	173x76.5
45mm	184x80.5
49mm	191x81.5

Resources

Related

Layout

Developer documentation

WidgetKit

Developing a WidgetKit strategy — WidgetKit

Videos

What's new in widgets

Bring widgets to life

Design widgets for visionOS

Change log

Date	Changes
January 17, 2025	Corrected watchOS widget dimensions.
June 10, 2024	Updated to include guidance for accented widgets in iOS 18 and iPadOS 18.
June 5, 2023	Updated guidance to include widgets in watchOS, widgets on the iPad Lock Screen, and updates for iOS 17, iPadOS 17, and macOS 14.
November 3, 2022	Added guidance for widgets on the iPhone Lock Screen and updated design comprehensives for iPhone 14, iPhone 14 Pro, and iPhone 14 Pro Max.

Copyright © 2025 Apple Inc. All rights reserved.